

Politechnika Świętokrzyska
Wydział Elektrotechniki,
Automatyki i Informatyki

Przemysłowe sieci komputerowe

WYKŁAD 3

Protokół Modbus

Paweł Strączyński

Katedra Urządzeń Elektrycznych i Automatyki · Politechnika Świętokrzyska



Cele wykładu

Po tym spotkaniu student potrafi…

1. Opisać model danych Modbus i dobrać funkcję do odczytu lub zapisu danego obszaru pamięci
2. Przeliczyć numer rejestru z dokumentacji na adres w ramce i odczytać wartości 32-bitowe
3. Zbudować i zdekodować ramkę Modbus RTU, ASCII i TCP — z CRC, LRC i nagłówkiem MBAP
4. Rozpoznać odpowiedź wyjątku i skonfigurować komunikację: QModMaster, S7-1200, Python

Pytanie na początek



Dokumentacja miernika: „napięcie L1 — rejestry 40101–40102, typ FLOAT32”.

Program odczytuje dwa rejestry i pokazuje: 17254 i 32768.

> Ile wynosi napięcie — i jak w ogóle zbudować to zapytanie?

Modbus — wprowadzenie



Rola i historia

Zastosowanie

Klient–serwer

Warianty protokołu

Sieci przemysłowe — rola

Gdzie w tym obrazie mieści się Modbus

Wymagania

Deterministyczność, niezawodność, integracja SCADA/PLC

Zastosowania

Czujniki, napędy, sterowniki, SCADA

Modbus to protokół warstwy aplikacji

- określa, jak zapytać urządzenie o dane i jak je zapisać (funkcje, adresy, format odpowiedzi),
- nie określa medium — działa na RS-485/RS-232 (RTU, ASCII) i na Ethernecie (TCP),
- nie określa znaczenia rejestrów — to opisuje dokumentacja konkretnego urządzenia.

Historia Modbus

Najstarszy, ale wciąż powszechnie stosowany



- Rok 1979, Modicon → Schneider Electric
- Otwarty standard, darmowa specyfikacja
- Najstarszy, ale wciąż powszechnie stosowany

1979
Modbus RTU i ASCII dla sterowników Modicon

1999
Modbus TCP — Modbus w sieci Ethernet

2006
specyfikacja „Modbus over Serial Line”

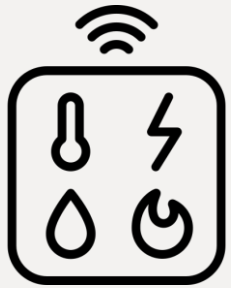
2018
Modbus/TCP Security (TLS, port 802)

2020
master/slave → klient/serwer w nazewnictwie

Prostota i bezpłatna specyfikacja sprawiły, że Modbus obsługuje niemal każde urządzenie automatyki — także jako „drugi” protokół obok PROFINET czy EtherCAT.

Zastosowanie

Urządzenia, które najczęściej rozmawiają po Modbusie



Czujniki i moduły IO

temperatura, wilgotność, moduły wejść analogowych i cyfrowych



Napędy, falowniki

zadawanie prędkości, odczyt prądu, kody błędów



Integracja z systemami SCADA

odczyt danych z wielu sterowników i urządzeń



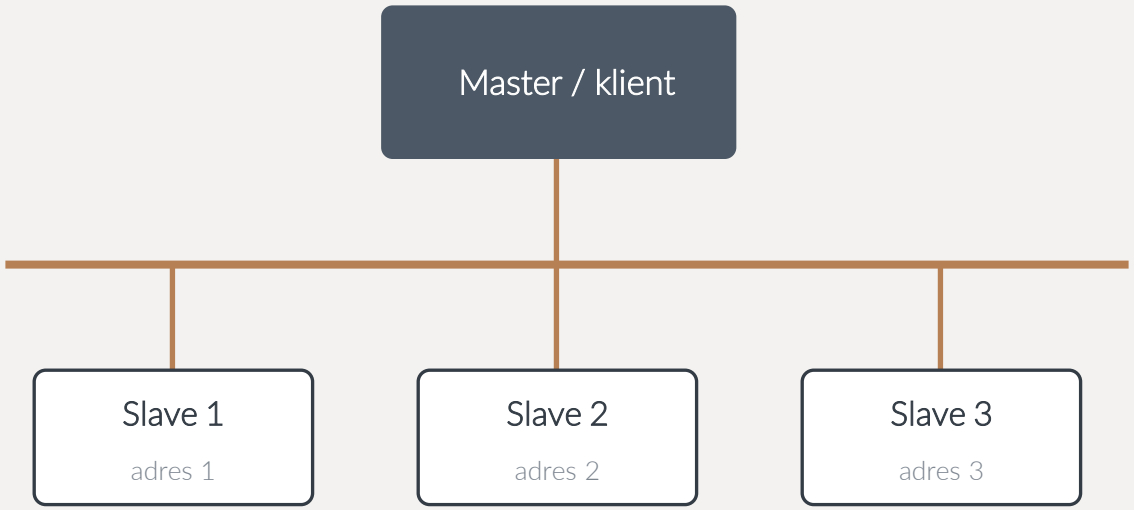
Komputery PC

aplikacje w Pythonie, C#, narzędzia diagnostyczne

Liczniki energii, analizatory sieci, sterowniki HVAC, falowniki, wagi — Modbus jest „wspólnym mianownikiem" automatyki.

Model komunikacji: klient–serwer (master–slave)

Jeden pyta, pozostali odpowiadają



① zapytanie do adresu 2

② odpowiada tylko Slave 2; pozostali ignorują ramkę

Slave nigdy nie nadaje z własnej inicjatywy

Modbus RTU / ASCII

Master i do 247 urządzeń slave (adresy 1–247) na magistrali.

Modbus TCP

Klient łączy się z serwerem (port 502); serwerów i klientów może być wielu.

Nazewnictwo

Od 2020 r. specyfikacja używa nazw klient/serwer — master = klient, slave = serwer.

Modbus a model OSI (ISO)

Ten sam protokół aplikacji na różnych warstwach niższych

Warstwa OSI	Modbus TCP
Aplikacji	Protokół Modbus
Prezentacji	-
Sesji	-
Transportu	TCP
Sieciowa	IP
Łącza danych	Ethernet (MAC)
Fizyczna	Medium: skrętka, światłowód

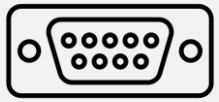
Warstwa OSI	Modbus RTU
Aplikacji	Protokół Modbus
Łącza danych	Modbus Serial Line: adres + CRC
Fizyczna	RS-485 / RS-232



Wspólna jest tylko warstwa aplikacji — funkcje i model danych. Dzięki temu bramka może tłumaczyć Modbus RTU na TCP bez zmiany treści zapytań.

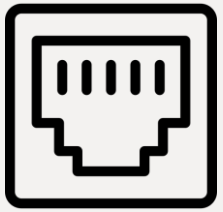
Warianty Modbus i standardy komunikacji

Wspólny rdzeń (PDU) w różnych opakowaniach (ADU)



Modbus RTU i Modbus ASCII

(RS-232, RS-422, RS-485)



Modbus TCP

(Ethernet)

PDU (Protocol Data Unit) — identyczne we wszystkich wariantach

kod funkcji	dane
1 B	0–252 B

ADU (Application Data Unit) — zależne od wariantu

RTU	adres 1 B	kod funkcji 1 B	dane 0–252 B	CRC 2 B
TCP	nagłówek MBAP 7 B	kod funkcji 1 B	dane 0–252 B	

II

Model danych i funkcje



Typy pamięci

Adresowanie rejestrów

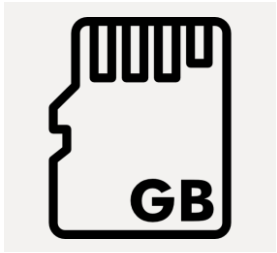
Kody funkcji

Typy danych

Typy pamięci

Cztery obszary danych — każdy z własnymi funkcjami

Protokół Modbus organizuje wymianę danych w oparciu o zdefiniowaną tablicę pamięci urządzenia (np. modułu I/O, serwonapędu). Obszary pamięci są podzielone według rodzaju danych i możliwości dostępu (tylko odczyt lub odczyt/zapis). Dostęp do każdego z obszarów pamięci możliwy jest za pomocą funkcji o określonym kodzie.



Typ pamięci	Zakres adresów	Rodzaj dostępu	Typ danych	Typowa zawartość
Coils	00001-09999	R/W	1 bit	wyjścia cyfrowe, polecenia start/stop
Discrete Inputs	10001-19999	RO	1 bit	wejścia cyfrowe, stany krańcówek
Input Registers	30001-39999	RO	16 bitów	wartości pomiarowe
Holding Registers	40001-49999	R/W	16 bitów	nastawy, parametry, także pomiary

W praktyce wielu producentów udostępnia wszystko w Holding Registers — decyduje mapa rejestrów w dokumentacji.

Numer rejestru a adres w ramce

Najczęstsze źródło pomyłek „o jeden”

40001 = obszar 4 (Holding Registers) + rejestr nr 1 → w ramce: funkcja 03, adres 0x0000

Dokumentacja	Funkcja	Adres w ramce (dziesiętnie)	Adres w ramce (hex)
40001	03 Read Holding Registers	0	0x0000
40101	03 Read Holding Registers	100	0x0064
30010	04 Read Input Registers	9	0x0009
00017	01 Read Coils / 05 Write Single Coil	16	0x0010
400001 (zapis 6-cyfrowy)	03 Read Holding Registers	0	0x0000

Niektóre dokumentacje podają od razu adres „od zera” (np. „rejestr 100”) — jeśli odczyt jest przesunięty o jeden rejestr, sprawdź tę konwencję.

Funkcje w Modbus

Kod funkcji mówi, co zrobić i w którym obszarze

Kod funkcji	Nazwa funkcji	Obszar pamięci	Maks. w jednej ramce
01	Read Coils	Coils	2000 bitów
02	Read Discrete Inputs	Discrete Inputs	2000 bitów
03	Read Holding Registers	Holding Registers	125 rejestrów
04	Read Input Registers	Input Registers	125 rejestrów
05	Write Single Coil	Coils	1 bit
06	Write Single Register	Holding Registers	1 rejestr
15	Write Multiple Coils	Coils	1968 bitów
16	Write Multiple Registers	Holding Registers	123 rejestry

Kody 15 i 16 w zapisie szesnastkowym to 0x0F i 0x10. Write Single Coil zapisuje wartość 0xFF00 (ON) lub 0x0000 (OFF).

Typy danych w rejestrach

Rejestr ma 16 bitów — większe wartości zajmują kilka rejestrów

Typ	Rejestrów	Zakres / uwagi
UINT16 / INT16	1	0…65 535 / −32 768…32 767
UINT32 / INT32	2	kolejność słów zależy od producenta
FLOAT32 (IEEE 754)	2	j.w. — najczęstsze źródło błędów
wartość skalowana	1	np. 2305 = 230,5 V (mnożnik 0,1)

Kolejność słów (word order)

ABCD — starsze słowo pierwsze (big-endian)
CDAB — młodsze słowo pierwsze (word swap)
W obrębie rejestru bajty są zawsze wysyłane od starszego (big-endian).

Odpowiedź na pytanie z początku

17254 **32768** → **0x4366** **0x8000** → 0x43668000 jako FLOAT32 → **230,5 V**

Gdyby odczyt dawał bzdury (np. $4{,}6 \cdot 10^{-41}$), należy zamienić kolejność rejestrów: CDAB zamiast ABCD.

Ramki RTU, ASCII i TCP



Ramka RTU i czasy ciszy

CRC-16

Ramka ASCII i LRC

Nagłówek MBAP

Modbus RTU

Binarny, zwarty, najpopularniejszy wariant szeregowy

W protokole Modbus RTU informacje przesyłane są w postaci binarnej.

Ramki rozpoczyna się zwłoką czasową co najmniej 3,5 razy dłuższą od czasu trwania pojedynczego znaku. Przerwy pomiędzy nadawaniem kolejnych znaków nie mogą być dłuższe niż 1,5 czasu trwania znaku.

Do weryfikacji poprawności przesyłanych informacji wykorzystywana jest suma kontrolna CRC (Cyclic Redundancy Check) — 16-bitowe słowo, przesyłane w postaci dwóch bajtów na końcu ramki.

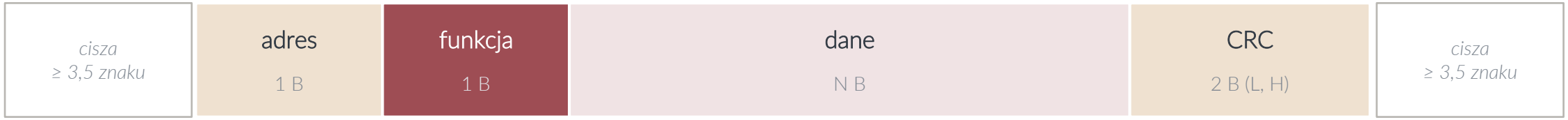
Słowo kontrolne jest wyliczane przez urządzenie nadawcze i odbiorcze oraz porównywane. Ewentualne rozbieżności świadczą o występujących przekłamaniach w transmitowanych danych.

W skrócie

- Działa na magistrali RS-485 (lub RS-232).
- Ramka zaczyna i kończy się „ciszą na linii” trwającą co najmniej tyle, ile czas przesłania 3,5 znaku.
- Dane są przesyłane w formie binarnej.
- Komunikacja master-slave (jednostka nadrzędna pyta — podrzędne odpowiadają).
- Często wykorzystywany w prostych systemach lokalnych.
- Poprawność ramki zabezpieczona przez CRC.

Budowa ramki Modbus RTU

Przykład z laboratorium: odczyt 2 rejestrów z urządzenia o adresie 1



ZAPYTANIE

01	03	00	00	00	02	C4	0B

adres 1 · funkcja 03 · od rejestru 0x0000 · 2 rejestry · CRC

ODPOWIEDŹ

01	03	04	0B	26	00	02	98	1D

liczba bajtów 4 · rejestr 0 = 0x0B26 = 2854 · rejestr 1 = 2

Długość ramki: maks. 256 bajtów (adres 1 + PDU 253 + CRC 2). **Adresy:** 0 — rozgłoszenie, 1–247 — urządzenia, 248–255 — zarezerwowane. **CRC:** młodszy bajt wysyłany jako pierwszy.

Zadanie: czasy ciszy i cykl odpytywania

9600 b/s, format 8-E-1 (11 bitów na znak), 10 urządzeń, odczyt 2 rejestrów z każdego

1 Czas znaku

11 bit / 9600 b/s

1,15 ms

2 Cisza 3,5 znaku

3,5 × 1,15 ms

4,0 ms

3 Jedno urządzenie

(8 + 9) znaków × 1,15 ms
+ 2 × 4,0 ms ciszy
+ ok. 5 ms reakcji urządzenia

≈ 32 ms

4 10 urządzeń

10 × 32 ms

≈ 0,32 s

Sterownik dowie się o zmianie w urządzeniu nr 10 nawet po ok. 0,3 s — przy 19 200 b/s czas skróci się mniej więcej o połowę.

Powyżej 19 200 b/s specyfikacja przyjmuje stałe czasy: 750 µs (1,5 znaku) i 1,75 ms (3,5 znaku).

Suma kontrolna CRC-16

Liczona przez nadawcę i odbiorcę — różnica oznacza przekłamanie

- wartość początkowa 0xFFFF
- wielomian 0xA001 dla każdego bajtu: XOR, potem 8 przesunięć w prawo
- wynik: 2 bajty, młodszy wysyłany pierwszy
- wykrywa wszystkie błędy 1- i 2-bitowe oraz serie błędów do 16 bitów

```
def crc16_modbus(data):
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
    return crc   # 0x0BC4 → wysyłamy C4 0B
```

Przykład: **`crc16_modbus([0x01, 0x03, 0x00, 0x00, 0x00, 0x02]) = 0x0BC4 → na końcu ramki: C4 0B`**

Modbus ASCII

Czytelny dla człowieka, ale dwa razy dłuższy

Modbus ASCII to rodzaj protokołu, w którym informacje przesyłane są w postaci znaków ASCII — dokładnie każdy bajt informacyjny to dwa znaki ASCII.

Protokół dopuszcza stosowanie dużego odstępu czasowego pomiędzy znakami — nawet do 1 sekundy.

Ramka komunikacyjna zawiera jasno zdefiniowane znaczniki początku i końca: zaczyna się znakiem dwukropka : (0x3A), kończy się znakami CR (Carriage Return — 0x0D) oraz LF (Line Feed — 0x0A).

Poprawność przesyłania ramki Modbus ASCII badana jest z wykorzystaniem kodu LRC (Longitudinal Redundancy Check).

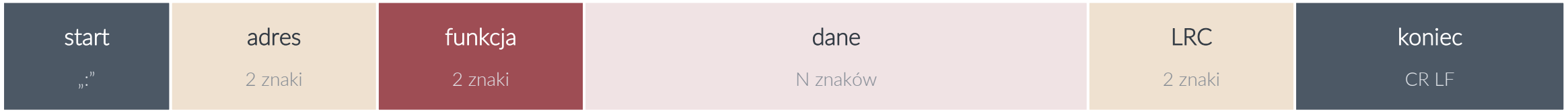
Wadą transmisji w trybie ASCII jest nadmiarowość danych koniecznych do przesłania tej samej informacji względem trybu RTU.

W skrócie

- Działa również na magistrali RS-485/RS-232.
- Dane są przesyłane jako ciągi znaków ASCII (czytelne dla człowieka).
- Ramka zaczyna się znakiem dwukropka : (0x3A), a kończy znakami CR (0x0D) oraz LF (0x0A).
- Wolniejszy od RTU, rzadziej stosowany.
- Także master-slave.
- Poprawność ramki zabezpieczona przez LRC.

Ramka Modbus ASCII i suma LRC

To samo zapytanie co w RTU — zapisane znakami



ZAPYTANIE (tekst przesyłany na linii)

```
:010300000002FA <CR><LF>
```

Jak liczy się LRC?

- suma bajtów (nie znaków!): 01 + 03 + 00 + 00 + 00 + 02 = 0x06
- uzupełnienie do dwóch: 0x100 − 0x06 = 0xFA
- zapis jako dwa znaki: „F” „A”

RTU a ASCII — ten sam PDU

- RTU: 8 bajtów = 8 znaków UART
- ASCII: 17 znaków (: + 14 znaków + CR LF)
- LRC słabsze niż CRC — wykrywa mniej typów błędów

Modbus TCP

Modbus w sieci Ethernet

W przypadku protokołu Modbus TCP ramka protokołu jest osadzona w standardowej ramce ethernetowej.

Port 502

Wersja działająca w sieci Ethernet, oparta na protokole TCP/IP (port 502).

Klient–serwer

Komunikacja klient-serwer (client-server), zamiast klasycznego master-slave.

Lokalnie i zdalnie

Umożliwia komunikację w sieciach lokalnych i zdalnych (np. SCADA ↔ PLC).

- brak CRC w ramce Modbus — integralność zapewniają TCP i suma FCS Ethernetu,
- klient może wysłać kilka zapytań bez czekania na odpowiedź — rozróżnia je numer transakcji,
- przez bramkę Modbus TCP ↔ RTU pole Unit ID wskazuje urządzenie na magistrali szeregowej.

Nagłówek MBAP

Modbus Application Protocol header — 7 bajtów przed PDU



ZAPYTANIE — ten sam odczyt 2 rejestrów, teraz przez Ethernet

00	01	00	00	00	06	01	03	00	00	00	02

ODPOWIEDŹ

00	01	00	00	00	07	01	03	04	0B	26	00	02

Transaction ID: numer nadany przez klienta, odsyłany w odpowiedzi

Protocol ID: zawsze 0 dla Modbus

Length: liczba bajtów po tym polu (Unit ID + PDU)

Unit ID: adres urządzenia za bramką; bez bramki zwykle 1 lub 0xFF

Porównanie wariantów Modbus

RTU, ASCII i TCP w jednej tabeli

Cecha	Modbus RTU	Modbus ASCII	Modbus TCP
Medium transmisji	RS-485/RS-232	RS-485/RS-232	Ethernet (TCP/IP)
Format danych	binarny	ASCII	binarny
Szybkość	wysoka	niska	wysoka
Tryb komunikacji	master-slave	master-slave	client-server
Kodowanie błędów	CRC16	LRC	brak w ramce Modbus — suma TCP i FCS Ethernetu
Granice ramki	cisza ≥ 3,5 znaku	znaki „:” oraz CR LF	pole Length w MBAP
Adresowanie	adres slave 1–247	adres slave 1–247	adres IP + Unit ID

IV

Transakcje i obsługa błędów



- Zapytanie–odpowiedź
- Rozgłoszenie
- Odpowiedzi wyjątków
- Timeout

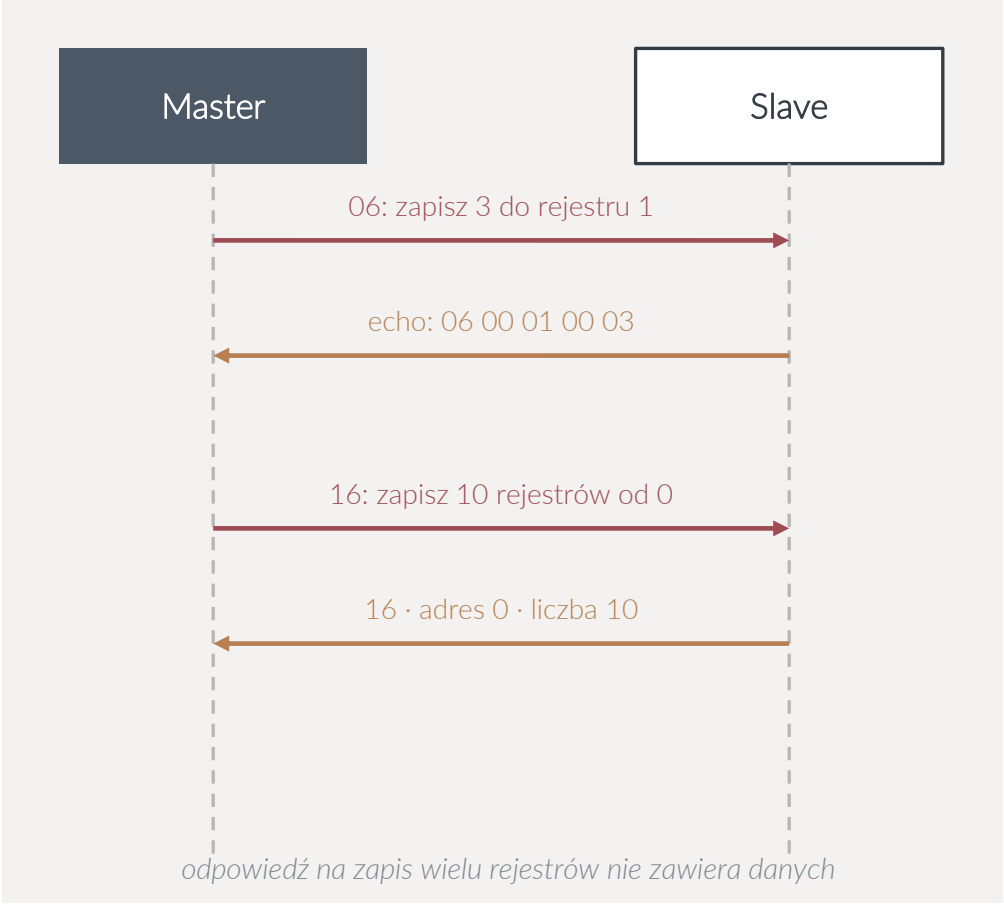
Rodzaje transakcji

Komunikację zawsze inicjuje urządzenie typu Master

Komunikacja w ramach protokołu Modbus zawsze inicjowana jest przez urządzenie typu Master. Transakcje można podzielić na: transakcja typu zapytanie-odpowiedź (Unicast) oraz transakcja typu rozgłoszenie (Broadcast).

W transakcji typu zapytanie-odpowiedź urządzenie typu Master łączy się z konkretnym urządzeniem typu Slave. W przypadku poleceń dotyczących odczytu stanu bitów lub rejestrów w odpowiedzi Slave odsyła odpowiednie informacje.

W przypadku wysłania przez urządzenie Master polecenia zapisu do rejestrów, Slave po wykonaniu operacji jako potwierdzenie odsyła ramkę do urządzenia typu Master. Przy zapisie większej liczby informacji jest to ramka skrócona zawierająca ilość zapisywanych rejestrów oraz adres pierwszego z nich.



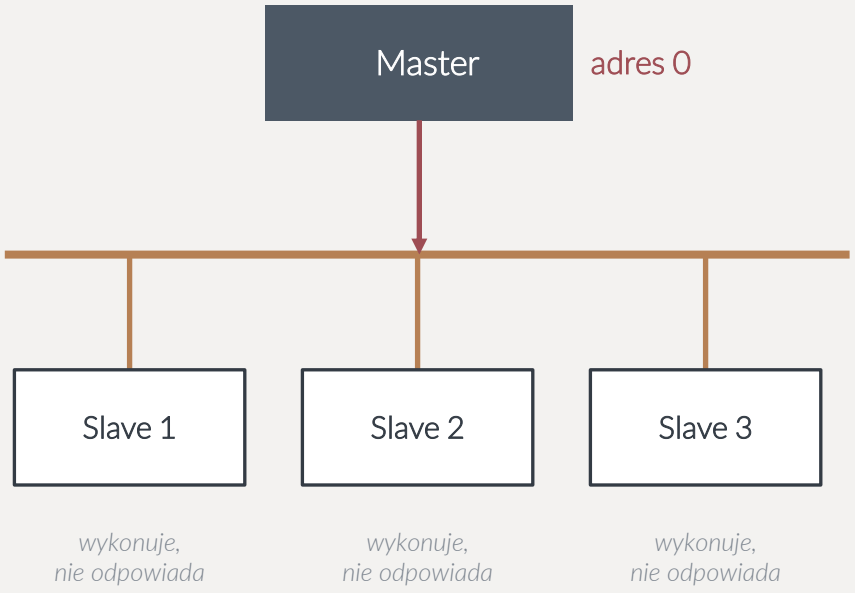
Transakcje broadcast (rozgłoszeniowe)

Jedno polecenie do wszystkich urządzeń na linii

Drugi typ transakcji, a więc transakcja typu rozgłoszeniowego, adresowana jest do wszystkich urządzeń na linii (zarezerwowanym adresem rozgłoszeniowym jest 0).

Slave'y po wykonaniu poleceń otrzymanych przez Master'a nie wysyłają ramki zwrotnej.

- tylko funkcje zapisu (np. 05, 06, 15, 16) — odczyt nie ma sensu, bo nikt nie odpowie
- master nie wie, czy polecenie dotarło — brak potwierdzenia
- po rozgłoszeniu master odczekuje (turnaround delay) przed kolejnym zapytaniem
- dotyczy tylko linii szeregowej; w Modbus TCP rozgłoszenia nie stosuje się



Przykład: jednoczesne zatrzymanie wszystkich napędów lub synchronizacja zegara

Odpowiedzi wyjątków

Urządzenie odebrało ramkę, ale nie może wykonać polecenia

ODPOWIEDŹ WYJĄTKU — zapytanie o nieistniejący rejestr

`01` `83` `02` `C0` `F1` funkcja 03 + 0x80 = 0x83 · kod wyjątku 02

Kod	Nazwa	Typowa przyczyna
01	Illegal Function	urządzenie nie obsługuje tej funkcji
02	Illegal Data Address	rejestr nie istnieje lub zakres wychodzi poza mapę
03	Illegal Data Value	niedozwolona wartość lub liczba rejestrów
04	Server Device Failure	błąd wewnętrzny urządzenia
06	Server Device Busy	urządzenie zajęte — ponów później
0B	Gateway Target Failed to Respond	bramka nie dostała odpowiedzi od urządzenia

Wyjątek ≠ timeout

Wyjątek: urządzenie żyje i odpowiada, ale zapytanie jest błędne — popraw adres lub funkcję.

Timeout: brak odpowiedzi — zły adres slave, prędkość, okablowanie albo błąd CRC (urządzenie ignoruje uszkodzone ramki).

V

Przykłady



Modbus RTU i QModMaster

Analiza ramek

S7-1200 jako serwer Modbus TCP

Python i Arduino

Modbus RTU — odczyt rejestrów i analiza ramek

Zakres i cel ćwiczenia

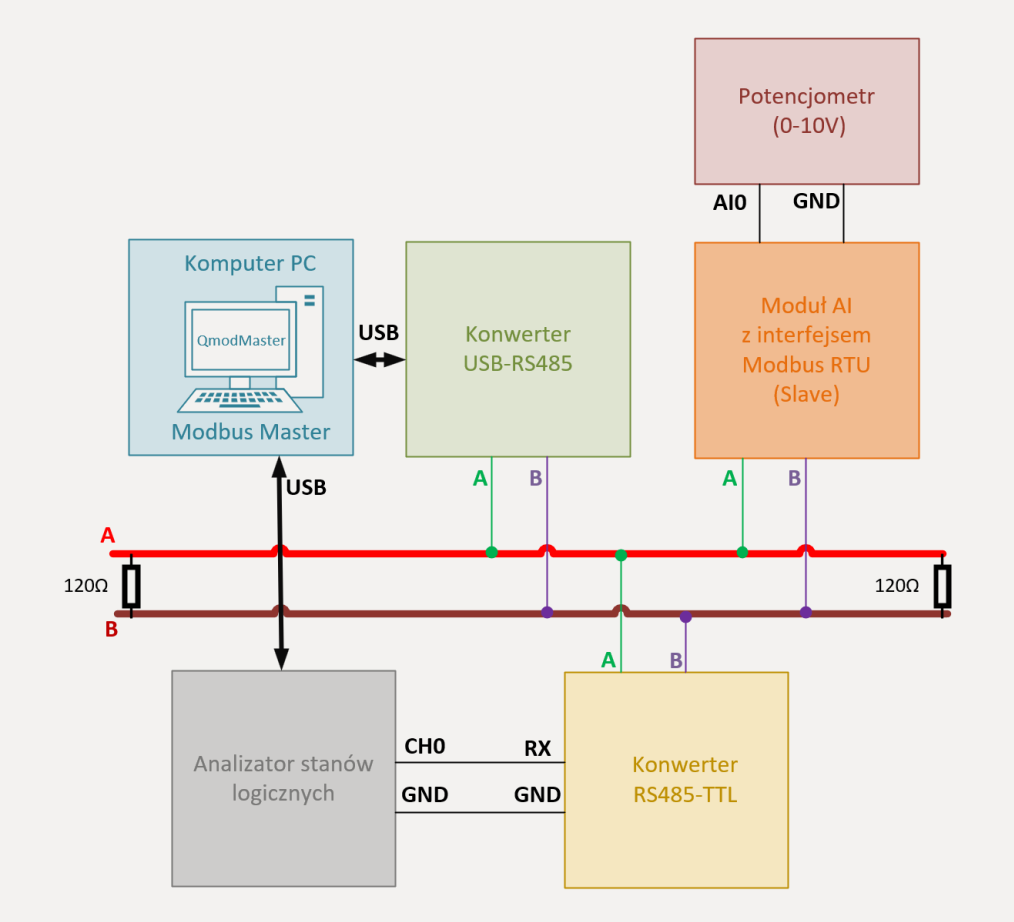
Poznanie praktycznego działania protokołu Modbus RTU poprzez odczyt wartości z modułu wejść analogowych (przez RS485), konfigurację klienta w programie QModMaster oraz analizę ramek komunikacyjnych za pomocą analizatora Saleae Logic.

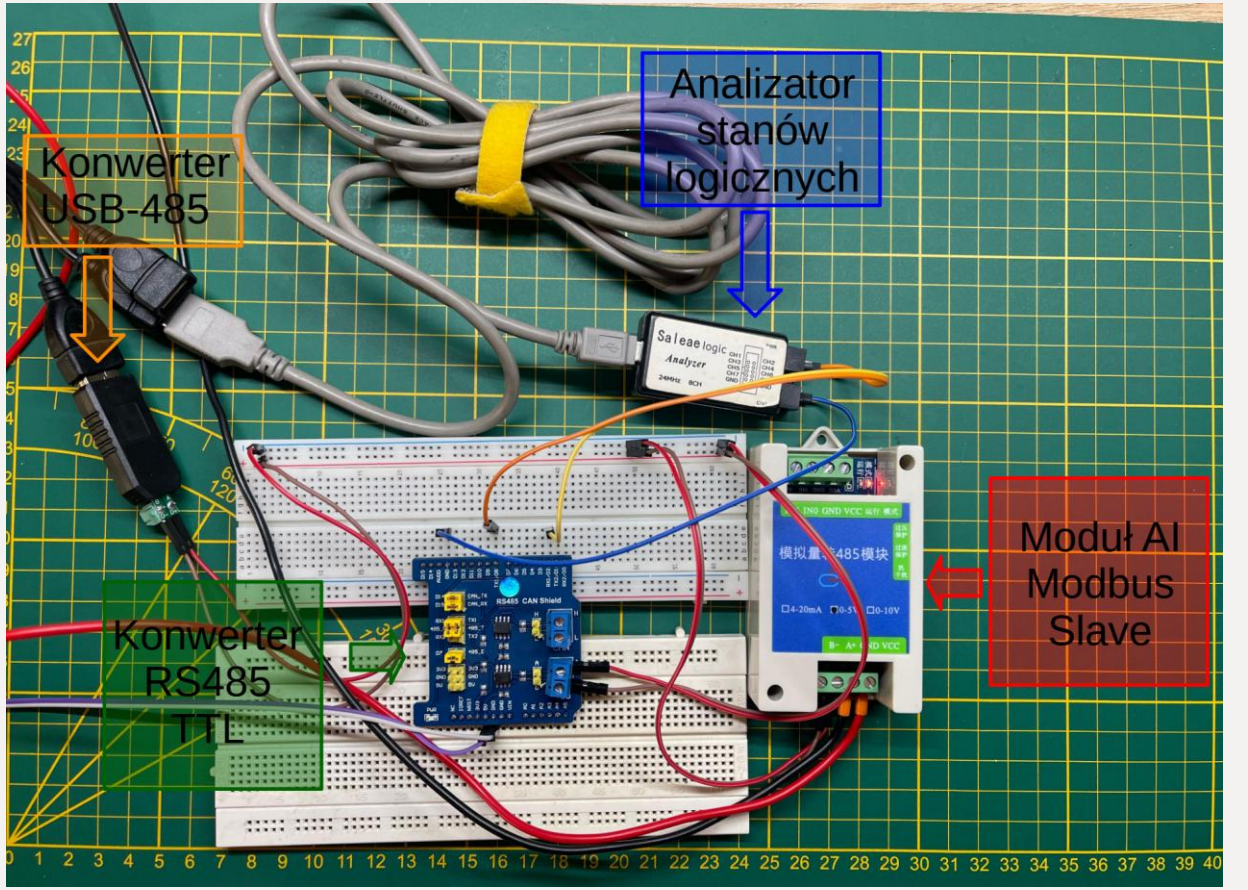
STANOWISKO LABORATORYJNE

1. Komputer PC z oprogramowaniem QModMaster.
2. Moduł wejść analogowych (slave) z interfejsem RS-485.
3. Konwerter USB–RS485.
4. Analizator stanów logicznych (Saleae Logic).
5. Przewody połączeniowe (z terminacją linii 120 Ω).

Schemat stanowiska laboratoryjnego

Połączenia i widok rzeczywistego stanowiska

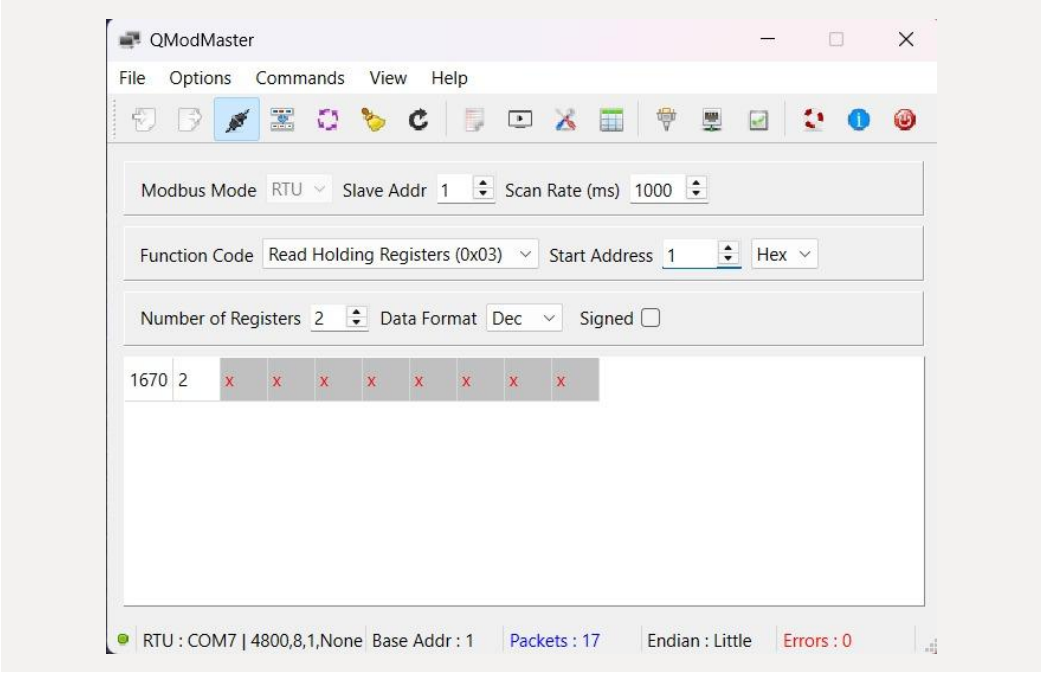




Konfiguracja QModMaster

Odczyt wejść analogowych modułu

1. Uruchomić QModMaster i wybrać port COM przypisany do konwertera USB–RS485.
2. Ustawić parametry transmisji zgodnie z dokumentacją modułu: 4800 bps (domyślnie), 8 bitów danych, brak parzystości, 1 bit stopu.
3. W polu Mode wybrać tryb RTU; w polu Slave ID wpisać adres urządzenia (domyślnie 1).
4. Function Code: 03 – Read Holding Registers; Address: 0 (0000H) dla kanału 1; Quantity: 2.
5. Kliknąć Send — wartości rejestrów pojawią się w formacie dziesiętnym i szesnastkowym.



Moduł zwraca wartość w zakresie 0–4095 (12 bitów). Dla wersji 0–5 V odczyt 1670 oznacza: $U = 1670 / 4096 \cdot 5\ V = 2{,}04\ V$

„Podsłuchiwanie” komunikacji analizatorem

Te same bajty, które poznaliśmy na slajdzie o ramce RTU



ZAPYTANIE

01 03 00 00 00 02 C4 0B

- 01 – adres slave (ID=1)
- 03 – funkcja: Read Holding Registers
- 00 00 – adres startowy rejestru: 0000h (kanał 1)
- 00 02 – liczba rejestrów: 2 (kanał 1 i kanał 2)
- C4 0B – CRC16 (low = C4, high = 0B) – poprawne dla bajtów 01 03 00 00 00 02

ODPOWIEDŹ

01 03 04 0B 26 00 02 98 1D

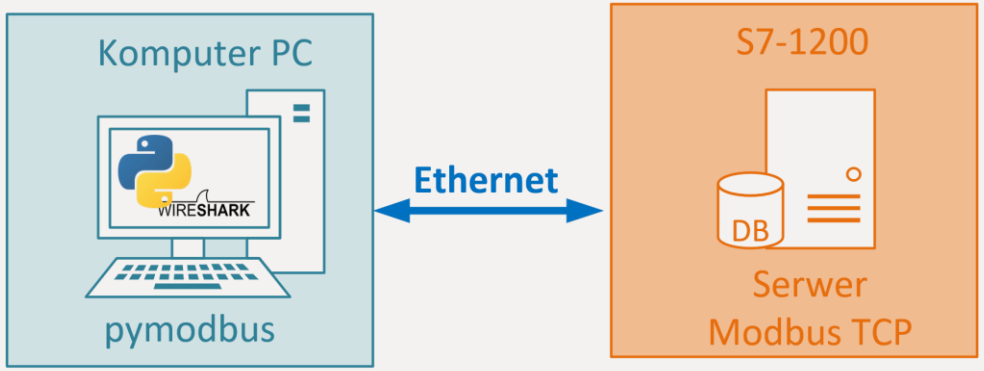
- 01 – adres slave
- 03 – echo funkcji
- 04 – liczba bajtów danych (2 rejestry × 2 bajty = 4)
- 0B 26 – rejestr 0000h (kanał 1) = 0x0B26 = 2854
- 00 02 – rejestr 0001h (kanał 2) = 0x0002 = 2
- 98 1D – CRC16

Modbus TCP — odczyt danych z S7-1200 w Pythonie

Zakres i cel ćwiczenia

Poznanie praktycznego działania protokołu Modbus TCP poprzez:

- konfigurację sterownika Siemens S7-1200 jako serwera Modbus TCP,
- mapowanie obszarów pamięci PLC na rejestry Modbus,
- odczyt wartości rejestrów w języku Python z wykorzystaniem biblioteki pymodbus,
- dekodowanie danych 16-bitowych, 32-bitowych oraz wartości zmiennoprzecinkowych,
- podstawową analizę ramek Modbus TCP w programie Wireshark.



Sprzęt i oprogramowanie

CPU S7-1200 (np. 1212C/1214C) + TIA Portal · PC w tej samej sieci Ethernet · Python 3.9+ z pakietem pymodbus (pip install pymodbus)

Konfiguracja serwera Modbus TCP na S7-1200 (1/2)

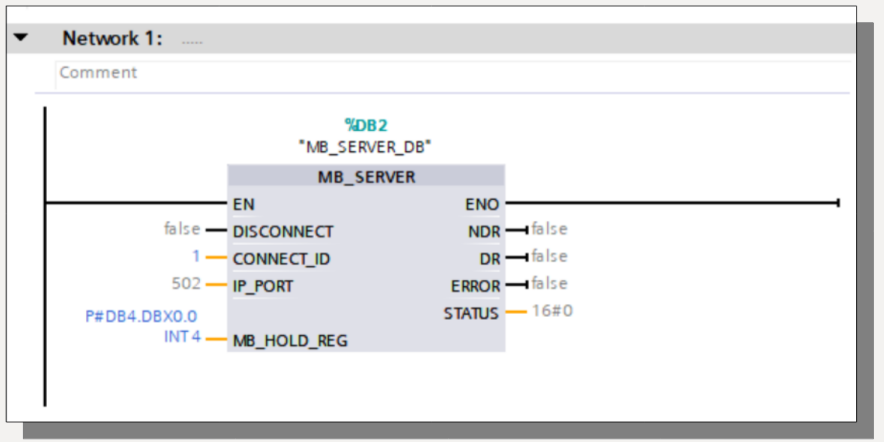
Sieć, blok MB_SERVER i blok danych

1. Konfiguracja sprzętowa i sieciowa

- W TIA Portal utworzyć projekt i dodać S7-1200 (w przykładzie CPU1212C AC/DC/RLY).
- Properties → Ethernet addresses: nadać CPU adres IP (np. 192.168.0.10) zgodny z podsiecią komputera.
- Sprawdzić, że komputer i sterownik są w tej samej sieci (test: ping).

Dodanie bloku MB_SERVER

- W OB1 wstawić blok MB_SERVER z instrukcji komunikacyjnych i utworzyć dla niego DB instancyjny.
- EN = TRUE, CONNECT_ID = 1, IP_PORT = 502.
- MB_HOLD_REG = wskaźnik na blok z rejestrami (P#DB4.DBX0.0), liczba rejestrów = 4.



Data_block_1

	Name	Data type	Offset	Start value
1	Static			
2	MB_HOLDING1	Int	0.0	0
3	MB_HOLDING2	Int	2.0	0
4	MB_HOLDING3	Int	4.0	0
5	MB_HOLDING4	Int	6.0	0
6	<Add new>			

2. Blok danych: cztery zmienne Int (MB_HOLDING1…4, offsety 0, 2, 4, 6) — każda to jeden rejestr (16 bitów).

Konfiguracja serwera Modbus TCP na S7-1200 (2/2)

Mapowanie rejestrów i test

3. Powiązanie bloku danych z MB_SERVER

W parametrze MB_HOLD_REG wskazać początek bloku danych (P#DB4.DBX0.0), w parametrze liczby rejestrów podać 4. Klient Modbus TCP może wtedy odczytywać i zapisywać wartości pod adresami 40001–40004.

Adres MB (HR)	Adres startowy (offset w DB)	Nazwa zmiennej	Typ danych
40001	DB4.DBW0 (offset 0.0)	MB_HOLDING1	INT (16 bit)
40002	DB4.DBW2 (offset 2.0)	MB_HOLDING2	INT (16 bit)
40003	DB4.DBW4 (offset 4.0)	MB_HOLDING3	INT (16 bit)
40004	DB4.DBW6 (offset 6.0)	MB_HOLDING4	INT (16 bit)

4. Wgrywanie i test

Wgrać konfigurację do sterownika, sprawdzić ping, odczytać rejestry klientem (QModMaster w trybie TCP lub Python).

Pozostałe obszary w MB_SERVER

Coils → wyjścia Q, Discrete Inputs → wejścia I, Input Registers → obraz wejść IW sterownika.

Odczyt w Pythonie (pymodbus)

Klient Modbus TCP w kilkunastu liniach

```
from pymodbus.client import ModbusTcpClient
import struct

client = ModbusTcpClient('192.168.0.10', port=502)
client.connect()

# 40001–40004 → adres 0, 4 rejestry
rr = client.read_holding_registers(0, count=4)
if rr.isError():
    print('Błąd:', rr)
else:
    print(rr.registers)          # np. [12, -5, 300, 0]
    hi, lo = rr.registers[0:2]
    f = struct.unpack('>f', struct.pack('>HH', hi, lo))[0]

client.close()
```

Adres od zera

40001 → read_holding_registers(0, …)

Sprawdzaj błędy

isError() zwraca wyjątek Modbus lub błąd połączenia

Liczby 32-bitowe

złóż dwa rejestry: >HH → >f (ABCD); przy CDAB zamień kolejność

Wersje biblioteki

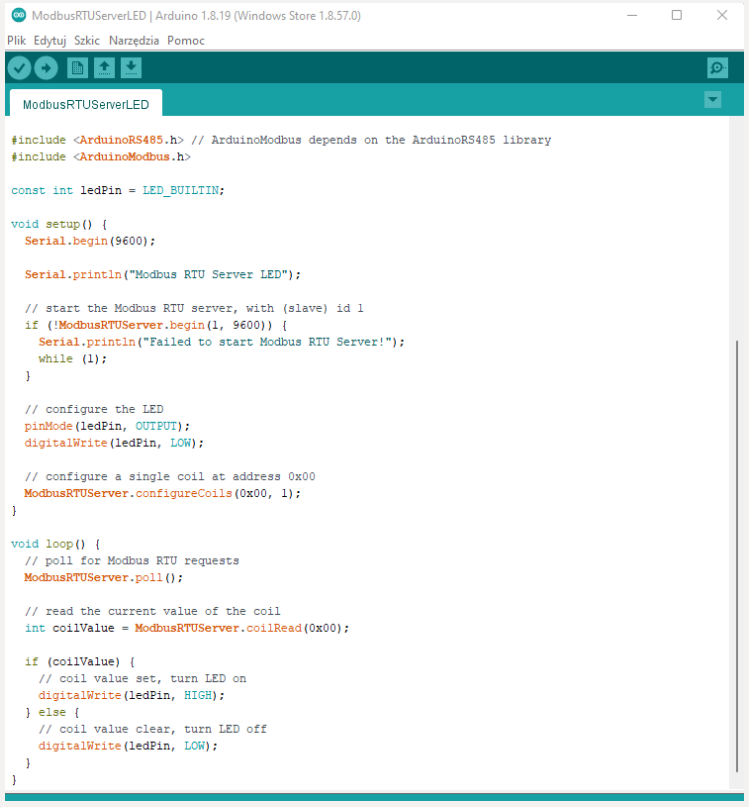
nazwy parametrów zmieniały się między wersjami 3.x — sprawdź dokumentację zainstalowanej wersji

Modbus w Arduino

Biblioteka ArduinoModbus: klient i serwer RTU



Klient: cyklicznie zapisuje cewkę w urządzeniu slave



Serwer: udostępnia cewkę, której stan steruje diodą LED

Pytania kontrolne

Sprawdź się — pytania w stylu kolokwium

1. Jakiej funkcji i jakiego adresu w ramce użyjesz, aby odczytać rejestr 30005?

2. Zdekoduj ramkę RTU: 11 03 00 6B 00 03 76 87.

3. Czym różni się odpowiedź wyjątku od timeoutu i co każde z nich sugeruje?

4. Dlaczego w ramce Modbus TCP nie ma CRC i co zastępuje adres slave?

Najważniejsze wnioski

Wracamy do pytania z początku: 17254 i 32768 to 230,5 V

1. Modbus to warstwa aplikacji: ten sam model danych i funkcje w RTU, ASCII i TCP

2. Numer z dokumentacji (40101) to nie adres w ramce (funkcja 03, adres 100)

3. Wartości 32-bitowe zajmują dwa rejestry — kolejność słów trzeba sprawdzić

4. Wyjątek wskazuje na błędne zapytanie, timeout — na połączenie i parametry transmisji

Materiały i kolejny wykład

Warto zajrzeć przed czwartym spotkaniem

Specyfikacje Modbus

modbus.org

Application Protocol Specification · Modbus over Serial Line · Messaging on TCP/IP.

Narzędzia

QModMaster · Wireshark

Klient Modbus RTU/TCP oraz analiza ramek Modbus TCP w sieci.

Python

pymodbus

Dokumentacja biblioteki i przykłady klienta oraz serwera.

Wykład 4

Przesyłanie danych pomiędzy sterownikami S7-1200: PUT/GET, TSEND_C / TRCV_C, I-Device.

Zostawiam chwilę na pytania końcowe.